1) Цель

   Знакомство с многопоточным программированием и методами синхронизации потоков средствами Windows API.

2) Задание

   a) С помощью таблицы вариантов заданий выбрать граф запуска потоков в соответствии с номером варианта. Вершины графа являются точками запуска/завершения потоков, дугами обозначены сами потоки. Длину дуги следует интерпретировать как ориентировочное время выполнения потока. В процессе своей работы каждый поток должен в цикле выполнять два действия: выводить букву имени потока в консоль; вызывать функцию computation() для выполнения вычислений, требующих задействования ЦП на длительное время. Эта функция уже написана и подключается из заголовочного файла lab3.h, изменять ее не следует.

   b) В соответствии с вариантом выделить на графе две группы с выполняющимися параллельно потоками. В первой группе потоки не синхронизированы, параллельное выполнение входящих в группу потоков происходит за счет планировщика задач. Вторая группа синхронизирована семафорами и потоки внутри группы выполняются в строго зафиксированном порядке: входящий в групу поток передает управление другому потоку после каждой итерации цикла (см. задачу производителя и потребителя). Таким образом потоки во второй группе выполняются в строгой очередности.

   c) С использованием средств Windows API реализовать программу для последовательно-параллельного выполнения потоков в ОС Windows. Запрещается использовать какие-либо библиотеки и модули, решающие задачу кроссплатформенной разработки многопоточных приложений (std::thread, Qt Thread, Boost Thread и т.п.), а также функции приостановки выполнения программы (например, Sleep(), SwitchToThread() и подобные). Для этого необходимо написать код в файле lab3.cpp:

      (i) Функция unsigned int lab3_thread_graph_id() должна возвращать номер графа запуска потоков, полученный из таблицы вариантов заданий.

      (ii) Функция const char* lab3_unsynchronized_threads() должна возвращать строку, состоящую из букв потоков, выполняющихся параллельно без синхронизации.

      (iii) Функция const char* lab3_sequential_threads() должна возвращать строку, состоящую из букв потоков, выполняющихся параллельно в строгой очередности друг за другом.

      (iv) Функция int lab3_init() заменяет собой функцию main(). В ней необходимо реализовать запуск потоков, инициализацию вспомогательных переменных (мьютексов, семафоров и т.п.). Перед выходом из функции

lab3_init() необходимо убедиться, что все запущенные потоки завершились. Возвращаемое значение: 0 - работа функции завершилась успешно, любое другое числовое значение - при выполнении функции произошла критическая ошибка.

(v) Добавить любые другие необходимые для работы программы функции, переменные и подключаемые файлы.

(vi) Создавать функцию main() не нужно. В проекте уже имеется готовая функция main(), изменять ее нельзя. Она выполняет единственное действие: вызывает функцию lab3_init().

(vii) Не следует изменять какие-либо файлы, кроме lab3.cpp. Также не следует создавать новые файлы и писать в них код, поскольку код из этих файлов не будет использоваться во время тестирования.

Интервалы с несинхронизированными потоками - dfgi

Интервалы с чередованием потоков - bcdg

3) Граф запуска потоков

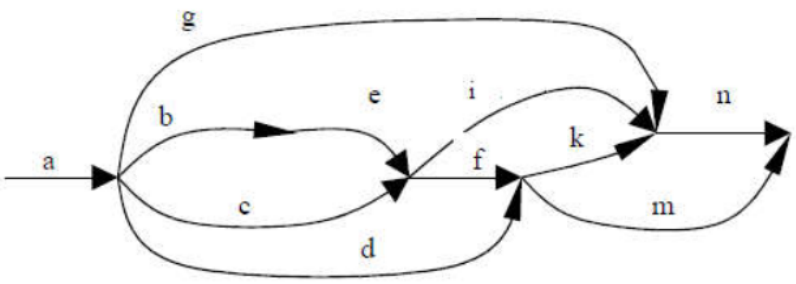


Рис. 1 – Граф запуска потоков

4) Результат выполнения работы

По результатам лабораторной работы была получена программа запуска потоков с синхронизацией средствами WinApi, которая прошла тестирование на работоспособность посредством автоматизированных тестов.

5) Исходный код

lab3.cpp

```
#include "lab3.h"
#include <windows.h>

//
// lab3 code should be located here!
//
```

```
#define DEFAULT_SEM_COUNT 1
#define THREADCOUNT 11

// Массив с потоками
// 0 - a
// 1 - b
// 2 - c
// 3 - d
// 4 - e
// 5 - f
// 6 - g
// 7 - i
// 8 - k
// 9 - m
// 10 - n
HANDLE Threads[THREADCOUNT];
// Мьютекс критической секции
HANDLE ThreadLock;
// Идентификатор для создания потока
DWORD ThreadID;
// Семафоры
HANDLE SemB, SemC, SemD, SemG, SemM;

// Сигнатуры функций потоков
DWORD WINAPI ThreadA(LPVOID);
DWORD WINAPI ThreadB(LPVOID);
DWORD WINAPI ThreadC(LPVOID);
DWORD WINAPI ThreadD(LPVOID);
DWORD WINAPI ThreadE(LPVOID);
DWORD WINAPI ThreadF(LPVOID);
DWORD WINAPI ThreadG(LPVOID);
DWORD WINAPI ThreadI(LPVOID);
DWORD WINAPI ThreadK(LPVOID);
DWORD WINAPI ThreadM(LPVOID);
DWORD WINAPI ThreadN(LPVOID);

unsigned int lab3_thread_graph_id()
{
    return 7;
}
```

```
const char* lab3_unsynchronized_threads()
{
    return "dfgi";
}

const char* lab3_sequential_threads()
{
    return "bcdg";
}

// Функции потоков
DWORD WINAPI ThreadA(LPVOID)
{
    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "a" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }

    Threads[1] = CreateThread(
        NULL,
        0,
        (LPTHREAD_START_ROUTINE)ThreadB,// Функция потока
        NULL,
        0,
        NULL);
    if (Threads[1] == NULL)
    {
        printf("CreateThread error: %d\n", GetLastError());
        return 1;
    }



    Threads[2] = CreateThread(
        NULL,
        0,
```

```
    (LPTHREAD_START_ROUTINE)ThreadC,// Функция потока
    NULL,
    0,
    NULL);
if (Threads[2] == NULL)
{
    printf("CreateThread error: %d\n", GetLastError());
    return 1;
}


Threads[3] = CreateThread(
    NULL,
    0,
    (LPTHREAD_START_ROUTINE)ThreadD,// Функция потока
    NULL,
    0,
    NULL);
if (Threads[3] == NULL)
{
    printf("CreateThread error: %d\n", GetLastError());
    return 1;
}


Threads[6] = CreateThread(
    NULL,
    0,
    (LPTHREAD_START_ROUTINE)ThreadG,// Функция потока
    NULL,
    0,
    NULL);
if (Threads[6] == NULL)
{
    printf("CreateThread error: %d\n", GetLastError());
    return 1;
}

ReleaseSemaphore(SemB, 1, NULL);

HANDLE ThreadHandles[4] = { Threads[1], Threads[2], Threads[3], Threads[6] };
```

```
    WaitForMultipleObjects(4,
        ThreadHandles,
        TRUE,
        INFINITE);

    return TRUE;
}

DWORD WINAPI ThreadB(LPVOID)
{
    for (int i = 0; i < 3; ++i) {
        WaitForSingleObject(SemB, INFINITE);
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "b" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
        ReleaseSemaphore(SemC, 1, NULL);
    }

    WaitForSingleObject(SemB, INFINITE);

    Threads[4] = CreateThread(
        NULL,
        0,
        (LPTHREAD_START_ROUTINE)ThreadE,// Функция потока
        NULL,
        0,
        NULL);
    if (Threads[4] == NULL)
    {
        printf("CreateThread error: %d\n", GetLastError());
        return 1;
    }

    ReleaseSemaphore(SemC, 1, NULL);
    ReleaseSemaphore(SemD, 1, NULL);
    ReleaseSemaphore(SemG, 1, NULL);
```

```
    return TRUE;
}

DWORD WINAPI ThreadC(LPVOID)
{
    for (int i = 0; i < 3; ++i) {
        WaitForSingleObject(SemC, INFINITE);
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "c" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
        ReleaseSemaphore(SemD, 1, NULL);
    }

    WaitForSingleObject(SemC, INFINITE);

    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "c" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }

    return TRUE;
}

DWORD WINAPI ThreadD(LPVOID)
{
    for (int i = 0; i < 3; ++i) {
        WaitForSingleObject(SemD, INFINITE);
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
```

```
        std::cout << "d" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
        ReleaseSemaphore(SemG, 1, NULL);
    }

    WaitForSingleObject(SemD, INFINITE);

    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "d" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }

    ReleaseSemaphore(SemG, 1, NULL);
    HANDLE Handles[2] = { Threads[2], Threads[4] }; // Потоки E C

    WaitForMultipleObjects(2,
        Handles,
        TRUE,
        INFINITE);

    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "d" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }

    return TRUE;
}

DWORD WINAPI ThreadE(LPVOID)
{
```

```
    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "e" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }

    return TRUE;
}

DWORD WINAPI ThreadF(LPVOID)
{
    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "f" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }

    return TRUE;
}

DWORD WINAPI ThreadG(LPVOID)
{
    for (int i = 0; i < 3; ++i) {
        WaitForSingleObject(SemG, INFINITE);
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "g" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
        ReleaseSemaphore(SemB, 1, NULL);
    }
```

```
WaitForSingleObject(SemG, INFINITE);

for (int i = 0; i < 3; ++i) {
    DWORD dwWaitResult = WaitForSingleObject(
        ThreadLock,   // Мьютекс
        INFINITE
    );
    std::cout << "g" << std::flush;
    ReleaseMutex(ThreadLock);
    computation();
}

HANDLE Handles[3] = { SemG, Threads[2], Threads[4]}; // Потоки E C и пауза
потока D

WaitForMultipleObjects(3,
    Handles,
    TRUE,
    INFINITE);

Threads[7] = CreateThread(
    NULL,
    0,
    (LPTHREAD_START_ROUTINE)ThreadI,// Функция потока
    NULL,
    0,
    NULL);
if (Threads[7] == NULL)
{
    printf("CreateThread error: %d\n", GetLastError());
    return 1;
}

Threads[5] = CreateThread(
    NULL,
    0,
    (LPTHREAD_START_ROUTINE)ThreadF,// Функция потока
    NULL,
    0,
    NULL);
```

```
if (Threads[5] == NULL)
{
    printf("CreateThread error: %d\n", GetLastError());
    return 1;
}

for (int i = 0; i < 3; ++i) {
    DWORD dwWaitResult = WaitForSingleObject(
        ThreadLock,   // Мьютекс
        INFINITE
    );
    std::cout << "g" << std::flush;
    ReleaseMutex(ThreadLock);
    computation();
}

HANDLE HandlesTwo[3] = { SemG, Threads[3], Threads[5] }; // Потоки D F и
пауза потока I

WaitForMultipleObjects(3,
    HandlesTwo,
    TRUE,
    INFINITE);

Threads[8] = CreateThread(
    NULL,
    0,
    (LPTHREAD_START_ROUTINE)ThreadK,// Функция потока
    NULL,
    0,
    NULL);
if (Threads[8] == NULL)
{
    printf("CreateThread error: %d\n", GetLastError());
    return 1;
}
Threads[9] = CreateThread(
    NULL,
    0,
    (LPTHREAD_START_ROUTINE)ThreadM,// Функция потока
    NULL,
```

```
        0,
        NULL);
    if (Threads[9] == NULL)
    {
        printf("CreateThread error: %d\n", GetLastError());
        return 1;
    }

    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "g" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }

    ReleaseSemaphore(SemM, 1, NULL);
    WaitForSingleObject(Threads[9], INFINITE);

    return TRUE;
}

DWORD WINAPI ThreadI(LPVOID)
{
    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "i" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }

    ReleaseSemaphore(SemG, 1, NULL);
    HANDLE Handles[2] = {Threads[3], Threads[5] }; // Потоки D F

    WaitForMultipleObjects(2,
        Handles,
```

```
        TRUE,
        INFINITE);

    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "i" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }


    return TRUE;
}

DWORD WINAPI ThreadK(LPVOID)
{
    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "k" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }

    return TRUE;
}

DWORD WINAPI ThreadM(LPVOID)
{
    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "m" << std::flush;
        ReleaseMutex(ThreadLock);
```

```
        computation();
    }

    HANDLE Handles[3] = { SemM, Threads[7], Threads[8] }; // Потоки G I K
    WaitForMultipleObjects(3,
        Handles,
        TRUE,
        INFINITE);

    Threads[10] = CreateThread(
        NULL,
        0,
        (LPTHREAD_START_ROUTINE)ThreadN,// Функция потока
        NULL,
        0,
        NULL);
    if (Threads[10] == NULL)
    {
        printf("CreateThread error: %d\n", GetLastError());
        return 1;
    }

    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "m" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }

    WaitForSingleObject(Threads[10], INFINITE);

    return TRUE;
}

DWORD WINAPI ThreadN(LPVOID)
{
    for (int i = 0; i < 3; ++i) {
        DWORD dwWaitResult = WaitForSingleObject(
```

```
            ThreadLock,   // Мьютекс
            INFINITE
        );
        std::cout << "n" << std::flush;
        ReleaseMutex(ThreadLock);
        computation();
    }

    return TRUE;
}

int lab3_init()
{
    // Создание мьютекса
    ThreadLock = CreateMutex(
        NULL,
        FALSE,  // Свободен изначально
        NULL    // Имя
    );
    if (ThreadLock == NULL)
    {
        printf("CreateMutex error: %d\n", GetLastError());
        return 1;
    }

    SemB = CreateSemaphore(
        NULL,
        0,      // начальное значение
        1,      // максимум
        NULL    // без имени
    );
    if (SemB == NULL) {
       std::cerr << "Ошибка создания семафора: " << GetLastError() << std::endl;
        return 1;
    }

    SemC = CreateSemaphore(
        NULL,
        0,      // начальное значение
        1,      // максимум
        NULL    // без имени
```

```
);
if (SemC == NULL) {
   std::cerr << "Ошибка создания семафора: " << GetLastError() << std::endl;
    return 1;
}

SemD = CreateSemaphore(
    NULL,
    0,      // начальное значение
    1,      // максимум
    NULL    // без имени
);
if (SemD == NULL) {
   std::cerr << "Ошибка создания семафора: " << GetLastError() << std::endl;
    return 1;
}

SemG = CreateSemaphore(
    NULL,
    0,      // начальное значение
    1,      // максимум
    NULL    // без имени
);
if (SemG == NULL) {
   std::cerr << "Ошибка создания семафора: " << GetLastError() << std::endl;
    return 1;
}

SemM = CreateSemaphore(
    NULL,
    0,      // начальное значение
    1,      // максимум
    NULL    // без имени
);
if (SemM == NULL) {
   std::cerr << "Ошибка создания семафора: " << GetLastError() << std::endl;
    return 1;
}

Threads[0] = CreateThread(
    NULL,
```

```
        0,
        (LPTHREAD_START_ROUTINE)ThreadA,// Функция потока
        NULL,
        0,
        NULL);

    if (Threads[0] == NULL)
    {
        printf("CreateThread error: %d\n", GetLastError());
        return 1;
    }

    WaitForSingleObject(Threads[0], INFINITE);

    CloseHandle(ThreadLock); // Очистка мьютекса
    // Очистка семафоров
    CloseHandle(SemB);
    CloseHandle(SemC);
    CloseHandle(SemD);
    CloseHandle(SemG);
    CloseHandle(SemM);

    return 0;
}
```

6) Выводы

В процессе выполнения лабораторной работы были получены практические навыки работы с потоками и их синхронизацией средствами Windows в операционной системе Windows 11.